

Sometimes a method is so cluttered with issues that it's easier to tear down the method and start fresh. And perhaps you have found an algorithm that's much simpler and more efficient. If this is the case, you should simply replace the old algorithm with the new one

As time goes on, your algorithm may be incorporated into a well-known library or framework and you want to get rid of your independent implementation, in order to simplify maintenance

The requirements for your program may change so heavily that your existing algorithm can't be salvaged for the task

How to Refactor

Make sure that you have simplified the existing algorithm as much as possible. Move unimportant code to other methods using Extract Method. The fewer moving parts in your algorithm, the easier it's to replace

Create your new algorithm in a new method. Replace the old algorithm with the new one and start testing the program

If the results don't match, return to the old implementation and compare the results. Identify the causes of the discrepancy. While the cause is often an error in the old algorithm, it's more likely due to something not working in the new one

When all tests are successfully completed, delete the old algorithm for good

Substitute Algorithm

مشکل (Problem)

می‌خواهی به الگوریتم موجود رو با یه الگوریتم جدید جایگزین کنی.

راه‌حل (Solution)

بدنه متدی که الگوریتم رو پیاده‌سازی می‌کنه با الگوریتم جدید جایگزین کن.

مثال از صفحه

قبل (الگوریتم قدیمی با if های تکراری):

csharp

```
string FoundPerson(string[] people)
{
    for (int i = 0; i < people.Length; i++)
    {
        if (people[i].Equals("Don")) return "Don";
        if (people[i].Equals("John")) return "John";
        if (people[i].Equals("Kent")) return "Kent";
    }
    return String.Empty;
}
```

بعد (الگوریتم جدید با List.Contains):

csharp

```
string FoundPerson(string[] people)
{
    List<string> candidates = new List<string>() {"Don", "John", "Kent"};

    for (int i = 0; i < people.Length; i++)
    {
        if (candidates.Contains(people[i]))
        {
            return people[i];
        }
    }
    return String.Empty;
}
```

چرا ریفتور کنیم؟ (Why Refactor) 

دلیل	توضیح
کد درهم و شلوغ	گاهی پاره کردن و از نو ساختن از ریفکتور تدریجی راحت‌تره
الگوریتم ساده‌تر و کارآمدتر	یه راه بهتر پیدا کردی
استفاده از کتابخونه	الگوریتمت توی یه کتابخونه معروف وجود داره ← پیاده‌سازی خودت رو حذف کن
تغییر نیازمندی‌ها	الگوریتم قدیمی دیگه جواب نمیده

How to Refactor (چطور انجام بدیم)

قدم	توضیح
۱	مطمئن شو الگوریتم موجود رو تا حد ممکن ساده‌سازی کردی (Extract Method)
۲	الگوریتم جدید رو توی یه متد جدید بساز
۳	الگوریتم قدیمی رو با جدید جایگزین کن و تست کن
۴	اگه نتایج تطابق نداشتن، برگرد به نسخه قدیمی و علت اختلاف رو پیدا کن
۵	وقتی همه تست‌ها موفق بودن، الگوریتم قدیمی رو برای همیشه حذف کن

مثال ساده

قبل:

csharp

```
public int CalculateSum(int[] numbers)
{
    ;int sum = 0
```

```

for (int i = 0; i < numbers.Length; i++)
    ;sum += numbers[i]
;return sum
    {

```

بعد (الگوریتم ساده‌تر با LINQ):

csharp

```

public int CalculateSum(int[] numbers)
    }
;()return numbers.Sum
    {

```

تمرین

این کد رو با **Substitute Algorithm** ریفتکتور کن (با یه روش ساده‌تر):

csharp

```

public class SearchHelper
    }

public bool ContainsItem(string[] items, string target)
    }

for (int i = 0; i < items.Length; i++)
    }

if (items[i] == target)
    ;return true
    {
        ;return false
    }
    {

```

راهنمایی: با LINQ یا `Array.Contains` جایگزین کن.

بفرست تا بررسی کنیم! 

خودت حلش کن

کد اصلاح شده با Substitute Algorithm

csharp

```
public class SearchHelper
{
    public bool ContainsItem(string[] items, string target)
    {
        ;return items.Contains(target)
    }
}
```

مقایسه

نسخه اصلاح	کد اصلی	
 ۱ خط	۹ خط	تعداد خطوط
 Contains (آماده و تست شده)	if + for	حلقه دستی
 یه خط واضح	باید بخونی بفهمی	خوانایی
 صفر	ممکنه (> < =)	احتمال باگ

Substitute Algorithm تموم شد!

بریم سراغ Consolidate Conditional Expression؟ متنش رو بفرست! 